



Universidad
Nacional de
General
Sarmiento

Instituto de Industria
Licenciatura en Sistemas

TRABAJO FINAL

Sistemas Operativos y Redes 2

Primer Semestre 2026

"Análisis de Logs de Seguridad con ELK Stack y Detección de Patrones con ML"

Alumno/a	Morinigo Franco
Legajo	41070609
Docente	Benjamin Chuquimango
Fecha de entrega	24-06-2026

Modalidad Individual — Obligatorio

Buenos Aires, Argentina — 2026

Índice de contenidos

1. Resumen ejecutivo
2. Planteamiento del problema
3. Pregunta de investigación y objetivos
4. Marco teórico
5. Metodología
 - 5.1 Enfoque metodológico
 - 5.2 Herramientas y justificación
6. Implementación y desarrollo
 - 6.1 Diseño de la solución
 - 6.2 Implementación por componente
 - 6.3 Análisis según modelo OSI o capas del SO
 - 6.4 Pruebas y verificación
7. Resultados y análisis
8. Conclusiones y trabajo futuro
9. Referencias bibliográficas

1. Resumen ejecutivo

El problema abordado se centra en la alta complejidad y el tiempo que requiere el análisis manual de grandes volúmenes de logs de seguridad mediante herramientas tradicionales como **ausearch**. La gran cantidad de registros generados por servicios críticos, como **auditd** y servidores web **Apache**, dificulta la detección oportuna de incidentes como accesos no autorizados o secuencias inusuales. Para resolver esto, se evalúa si un pipeline ELK centralizado, complementado con el módulo de Machine Learning de **Elasticsearch**, es capaz de centralizar, indexar y analizar estos eventos en tiempo real, identificando anomalías de manera automatizada y eficiente frente a los métodos manuales. La metodología aplicada fue utilizar una máquina virtual con **Docker** Compose para levantar los servicios del ELK Stack, **filebeat** para tener un agente más ligero que logstash en el servidor. Los resultados obtenidos fueron notablemente distintos debido a que con el análisis manual el tiempo que se demora en encontrar patrones anómalos es muy superior al tiempo que se tarda con el ELK Stack, los mismos demostraron que la plataforma detectó exitosamente todos los eventos maliciosos, debido a que eran eventos para hacer ruido y así ser más fácil de detectar ante el análisis manual, en aproximadamente 35 segundos, en contraste con el análisis manual demandó casi una hora para hallar los mismos patrones a pesar de tener mucha información sobre el ruido generado por uno mismo. La principal contribución de este trabajo es demostrar lo viable que es reemplazar la inspección de logs aislados por un sistema centralizado e inteligente, reduciendo drásticamente los tiempos de respuesta ante incidentes y revelando patrones de comportamiento anómalo que escapan a las reglas tradicionales.

Palabras clave: *Ausearch, Auditd, Elasticsearch, Docker, Filebeat.*

2. Planteamiento del problema

En las infraestructuras actuales nos encontramos con volúmenes de logs que sobrepasan la capacidad humana de análisis, no seríamos capaces de realizar un análisis sobre miles de logs generados por hora, sumado a esto, dichos logs del sistema no se encuentran con una estructura fácil de interpretar, un analista con experiencia puede estar más familiarizado con la lectura de logs, sin embargo, no significa que sea capaz de realizar un análisis profundo en un tiempo reducido para poder reducir el impacto ante ataques. Utilizando `auditd` y servidores web como Apache, generamos una cantidad masiva de logs continuos, en una empresa u organización esta se intensifica de forma inmensa, Argentina registró 5700 millones de intentos de ciberataque durante 2025, esto quiere decir que son millones a día, Fortinet (Empresa multinacional de Ciberseguridad de Estados Unidos) advierte que la inteligencia artificial acelera y vuelve más sofisticado al cibercrimen. Los ciberataques se realizan de forma automatizada y reduce los tiempos que le toma a los ciberatacantes explotar vulnerabilidades.

Para realizar el análisis de forma manual de estos volúmenes de logs de seguridad, podríamos utilizar herramientas de línea de comando como `ausearch` y `aureport`, sin embargo, como veremos más adelante esto es ineficiente y muy lento en comparación a utilizar el pipeline ELK Stack con el módulo de Machine Learning de Elasticsearch. Como mencionado anteriormente, los analistas se ven sobrepasados por la cantidad de información que deben procesar.

El tiempo que tardamos en analizar logs de forma manual es muy valioso debido a que cada hora que pasamos investigando el problema se puede estar intensificando e incluso explotando más vulnerabilidades por lo que puede llevar a un riesgo muy alto en la organización afectada. Según el autor Pozzi (2021) existe una tendencia hacia el “aumento de automatizaciones e inteligencia artificial para enfrentar ciberataques, principalmente debido a la falta de capital humano especializado”, con esto podemos ver que no solo existe un volumen masivo de logs que hace imposible el análisis manual, sino que también hay una escasez de analistas capaces de poder realizar el trabajo necesario para este análisis.

Utilizando `auditd` para la creación de reglas podemos dirigir estos logs hacia el pipeline ELK Stack y evitar la lectura de logs desde la terminal, allí veremos logs de forma ordenada pero poco intuitiva ante personas no especializadas en el área, además para poder tener logs más legibles debemos aplicar filtros, si utilizamos el Stack podemos utilizar un agente ligero como Filebeat para que sea el encargado de leer los logs y enviarlos hacia Logstash para que al utilizar Grok tengamos datos más estructurados que se pueden enviar hacia Elasticsearch donde al tokenizar se envían hacia Kibana, esto simplemente lo configuraremos una vez y podríamos utilizar para el diseño de Dashboards, como también la aplicación de filtros mediante KQL (Kibana Query Language). Notaremos que hay una diferencia muy grande ya que al tener generado los dashboards junto con los filtros se pueden visualizar de forma más rápida el flujo de logs o comportamientos anómalos dentro de nuestro servidor.

Aplicando el módulo de Machine Learning de Elasticsearch el mismo se encargará de tener un comportamiento de lo que es considerado “normal” para que ante alguna anomalía poder generar distintos tipos de alertas, uno de los posibles ejemplos es la utilización de mails para avisar de dicha anomalía.

3. Pregunta de investigación y objetivos

3.1 Pregunta de investigación

¿Puede un pipeline ELK Stack centralizar, indexar y analizar esos logs en tiempo (casi) real, y puede el módulo de ML de Elasticsearch detectar automáticamente patrones anómalos (picos de intentos de autenticación fallida, acceso a rutas no habituales de Apache, secuencias de syscalls inusuales) que el análisis manual con asearch tardaría horas en encontrar?

3.2 Objetivo general

Implementar y evaluar una arquitectura centralizada basada en el ELK Stack y el módulo de Machine Learning de Elasticsearch para el análisis automatizado de registros de seguridad, para así poder cuantificar la reducción en los tiempos que se demora en detectar incidentes en comparación con la inspección forense manual.

3.3 Objetivos específicos

1. Desplegar el stack ELK mediante Docker Compose y el agente ligero Filebeat y los pipelines de Logstash para la ingesta y estructuración de logs de auditd y Apache en tiempo real.
2. Diseñar dashboards de seguridad en Kibana para centralizar las métricas mediante visualizaciones interactivas de eventos, intentos de acceso y alertas.
3. Aplicar el módulo de Machine Learning sobre los datos ingresados para identificar de forma automática anomalías y patrones de ataque generados en un entorno de laboratorio.
4. Medir y comparar el tiempo operativo que toma detectar eventos de seguridad conocidos de forma manual utilizando asearch y aureport frente a la detección automática del stack ELK.

4. Marco teórico

En un estudio reciente, Robbani et al. (2025) evaluaron la integración de herramientas de código abierto para la detección de ciberataques, específicamente combinando el sistema de detección de intrusos Snort con la plataforma ELK Stack (Elasticsearch, Logstash y Kibana). En su investigación, simulaban ataques de fuerza bruta (SSH), escaneo de puertos y Ping Flood, demostrando que el ELK Stack es altamente eficaz para procesar y visualizar estos eventos en tiempo real a través de dashboards centralizados.

Esta investigación aporta una validación sobre la eficacia del ELK Stack como solución para el monitoreo de seguridad. Los autores destacan que herramientas tradicionales tienen dificultades para identificar patrones complejos, mientras que Elasticsearch y Logstash permiten indexar grandes volúmenes de datos en tiempo real. Este argumento respalda directamente la decisión de utilizar el ELK Stack en este proyecto para centralizar los registros masivos, comprobando que es la arquitectura ideal para dar soporte a la inspección forense.

Por otro lado, Yang et al. (2021) utilizan un sistema automatizado de detección y análisis de ciberataques con el Stack ELK para la recolección y visualización de registros de red (NetFlow Logs). Para clasificar y predecir el tráfico malicioso, ellos extraen los registros almacenados en la plataforma y los evalúan comparando algoritmos predictivos de Machine Learning (XGBoost) y redes neuronales profundas (RNN y DNN), con los cuáles logran resultados altamente precisos.

Con este documento podemos encontrarnos que nos aporta una base directa sobre la viabilidad y el éxito de combinar la plataforma ELK con Machine Learning para la detección de anomalías de seguridad. Los autores pudieron demostrar que, frente al alto costo de los sistemas analíticos comerciales, una plataforma de código abierto apoyada por modelos predictivos logra una precisión superior al 96% en la identificación de patrones de ataque en grandes volúmenes de logs. Este hallazgo justifica firmemente la decisión técnica de la arquitectura elegida en el presente proyecto.

En su reciente investigación, Sekar et al. (2024) evalúan el rendimiento de los sistemas de recolección de registros de auditoría en Linux, focalizándose en herramientas tradicionales como auditd y sysdig. Los autores demuestran que, bajo cargas de trabajo moderadas e intensas, estas herramientas imponen una sobrecarga altísima (ralentizando los sistemas), sufren una pérdida masiva de eventos (data loss) y presentan una amplia ventana de vulnerabilidad que permite a los atacantes borrar los logs (log tampering) localmente antes de que se almacenen de forma segura.

Este paper nos proporciona el sustento para la comparativa operacional (Baseline) y la centralización de logs de este proyecto. Sekar et al. (2024) prueban de forma académica que depender del almacenamiento local y la lectura de auditd frente a amenazas persistentes es ineficiente e inseguro. Esto justifica la necesidad de extraer los logs mediante agentes como Filebeat y centralizarlos en un Stack ELK, evitando la pérdida de evidencia por manipulación local de los atacantes y mitigando el factor de lentitud del administrador del sistema.

En su investigación, Anjum y Chowdhury (2024) exploran cómo la automatización mediante Inteligencia Artificial está revolucionando los procesos de auditoría en ciberseguridad. Los autores argumentan que las metodologías tradicionales basadas en reglas estáticas son ineficientes ante las amenazas actuales. Advierten que los enfoques manuales son intensivos en mano de obra y consumen demasiado tiempo.

Por ello, proponen la integración de algoritmos de Machine Learning para analizar grandes volúmenes de datos en tiempo real, identificar anomalías y automatizar tareas rutinarias de auditoría.

Así mismo, nos aporta el sustento teórico exacto para la contribución operacional principal de este proyecto. Anjum y Chowdhury (2024) afirman que “las metodologías de auditoría tradicionales son a menudo intensivas en mano de obra, consumen mucho tiempo y recursos, lo que hace difícil para las organizaciones realizar auditorías exhaustivas y oportunas”. Esto justifica a la perfección la métrica comparativa de rendimiento, demostrando con validez académica por qué buscar eventos de seguridad a mano con herramientas consola resulta operativamente inviable, y por qué centralizarlos en un stack automatizado con módulos de Machine Learning soluciona este cuello de botella.

Como último recurso a utilizar en el marco teórico, nos basaremos en la investigación sobre el análisis de datos de registros en línea, Skopik et al. (2021) proponen un enfoque basado en aprendizaje automático para la detección proactiva de anomalías cibernéticas, superando las limitaciones del análisis forense retrospectivo. Los autores estructuran un pipeline de cuatro pasos que incluye la agrupación incremental de eventos, la generación de plantillas, la creación de analizadores (parsers) y el modelado de comportamiento. Utilizando logs de acceso de servidores web (como Apache) como caso de estudio, demuestran cómo los algoritmos de Machine Learning pueden identificar ataques sigilosos al detectar picos de frecuencia, nuevos valores de agentes de usuario y desviaciones en la correlación de solicitudes HTTP (como anomalías en secuencias de métodos GET y POST).

Este documento aporta una justificación teórica y metodológica central para la ingesta y análisis de eventos web de este proyecto. Skopik et al (2021) validan académicamente que cuando se trata de sistemas empresariales donde los volúmenes de logs ascienden a millones por día, la inspección manual o el simple uso de listas de bloqueo (reglas estáticas) es operativamente imposible y propenso a errores. Esta premisa fundamenta de forma directa la decisión técnica de centralizar los logs de acceso de Apache en el pipeline ELK, estructurar los datos sintácticamente (mediante filtros Grok) y delegar el descubrimiento de anomalías ocultas o de ataques en ráfaga a modelos automatizados de Machine Learning.

4.1 Related Work

En los papers citados nos encontramos que ciertas limitaciones o diferencias en cuanto al trabajo realizado en este proyecto, por un lado tenemos que alguno de ellos su capacidad de detección depende estrictamente de reglas predefinidas estáticas mediante Snort. Además, su enfoque es netamente de monitoreo de tráfico de red y visualización en Kibana, teniendo una falta de un componente de detección de anomalías automatizado e inteligente. El trabajo supera esta limitación al incorporar el módulo de Machine Learning, buscando automatizar la identificación de patrones anómalos a nivel de sistema operativo (mediante auditd) y servidor web (Apache), sin depender exclusivamente de reglas estáticas.

También existen limitaciones respecto al centro de los análisis en los que se centran puramente en el tráfico de la capa de red (logs de NetFlow) y usa algoritmos de clasificación supervisada que requieren entrenamiento previo con datos de ataques ya etiquetados, en comparación con el presente trabajo que aplica métodos de detección de anomalías no supervisados (como Elastic ML) y enfoca su análisis en los registros de seguridad generados directamente a nivel del sistema operativo (auditd) y del servidor web (Apache), cubriendo una capa de infraestructura diferente.

En otro paper se reemplaza la base de datos de auditd utilizando una nueva herramienta de recolección de datos a nivel de kernel y evitar la caída del rendimiento, en este proyecto, no se buscó modificar el mecanismo de captura del sistema operativo, sino que utiliza las fuentes auditd y Apache, y sitúa la innovación en la capa de análisis y detección automatizada, utilizando Machine Learning para identificar anomalías en los registros una vez centralizados.

Por último, en otros papers tenemos un enfoque sumamente conceptual, gerencial y ético. Los autores abordaron la problemática desde la teoría de la gestión de riesgos y el sesgo de la IA, pero no presentan una arquitectura técnica específica ni evalúan herramientas concretas. En contraste, en este trabajo se utilizaron estos conceptos junto con una implementación de laboratorio totalmente técnica y evaluable, construyendo el pipeline exacto mediante el ELK Stack para analizar registros crudos del sistema operativo Linux (auditd) y servidores web (Apache), además se optó por utilizar una solución basada en estándares de la industria, utilizando los filtros predefinidos de Logstash para el análisis estructural y aplicando directamente el motor integrado de Elastic ML para la detección de anomalías, enfocándose en la implementación y escalabilidad operativa práctica.

5. Metodología

5.1 Enfoque metodológico

En el presente trabajo la implementación fue a base de un laboratorio experimental, utilizando un Ubuntu Server, al inicio se desplegó el Stack ELK (Elasticsearch, Logstash y Kibana) con Docker Compose, se verificó la conexión entre ellos y luego se instaló Filebeat como agente ligero para la recolección de logs de los servicios de auditd y Apache para enviárselos a Logstash. A continuación, se realizó una inyección controlada de eventos de seguridad conocidos, como accesos no autorizados, intentos de inicio de sesión o accesos a archivos no autorizados), con ello se pudo realizar una evaluación de la respuesta del sistema.

Este enfoque es el más adecuado para responder a la pregunta de investigación planteada, debido a que la evaluación de una herramienta de monitorización y su comparación de rendimiento no pueden realizarse de manera puramente teórica ni mediante simulaciones abstractas. El diseño experimental permite cuantificar el tiempo que demora la detección de anomalías utilizando el módulo de Machine Learning frente a la inspección forense manual tradicional con herramientas de consola. De esta forma los resultados obtenidos se basan en métricas reales de consumo de recursos y tiempo de respuesta, garantizando que las conclusiones sobre la viabilidad de la automatización del análisis de logs sean verificables y aplicables a entornos de producción.

5.2 Herramientas y justificación

Herramienta Tecnología	Propósito en el trabajo	Justificación de la elección
Elasticsearch	Motor central de búsqueda, indexación y almacenamiento de registros (logs).	Se eligió por su alta escalabilidad y su capacidad de buscar en grandes volúmenes de datos en tiempo casi real gracias a su estructura de índice invertido. Constituye el núcleo de la arquitectura centralizada, reemplaza la búsqueda secuencial en archivos de textos locales.
Logstash	Pipeline de ingesta, procesamiento y transformación de datos en tiempo real.	Resulta indispensable para estructurar registros complejos mediante el uso de filtros Grok. Se optó por esta herramienta por esta capacidad nativa para normalizar y enriquecer los datos de múltiples fuentes antes de su indexación.
Kibana	Interfaz gráfica web para la exploración de datos, diseño de dashboards y gestión de alertas/ML.	Es la solución estándar del stack para visualizar la información indexada. Se seleccionó porque permite centralizar la monitorización y exponer los patrones anómalos detectados por

Herramienta Tecnología	/	Propósito en el trabajo	Justificación de la elección
			el módulo de Machine Learning, superando las limitaciones de herramientas como aureport.
Filebeat		Agente ligero encargado de leer los archivos de logs locales y enviarlos hacia Logstash.	A diferencia de Logstash, que requiere la JVM y presenta un alto consumo de recursos, Filebeat es extremadamente ligero. Se eligió para recolectar los datos en el servidor de origen minimizando el impacto en la CPU y RAM, asegurando que el proceso de auditoría no reduzca el rendimiento del sistema operativo.
auditd		Fuente de generación de logs de seguridad a nivel del sistema operativo y llamadas al sistema (syscalls).	Es el framework de auditoría estándar en entornos Linux. Se seleccionó porque proporciona una visibilidad profunda sobre eventos críticos.
Apache2		Servidor web utilizado como fuente de generación de logs de acceso web y errores HTTP.	Se usó para simular un servicio crítico de producción realista que genera un flujo constante de tráfico.
Docker Compose		Despliegue centralizado del stack ELK completo en un entorno de Laboratorio.	Se optó por un despliegue mediante contenedores frente a la instalación nativa para garantizar la reproducibilidad, portabilidad y aislamiento del entorno. Aunque introduce un ligero overhead de recursos, permite orquestar el stack simultáneamente mediante un único archivo YAML, evitando conflictos de dependencias en el sistema operativo host.

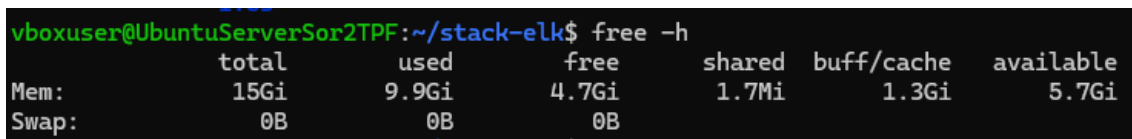
6. Implementación y desarrollo

6.1 Diseño de la solución

El diseño de la solución propuesta se basa en un pipeline de ingesta, procesamiento y visualización de datos, desplegado integralmente a través de contenedores mediante Docker Compose. La arquitectura se estructuró para tener un bajo impacto en el servidor origen y delegar la carga de procesamiento al nodo central.

El flujo de datos se realizó empezando por la recolección con Filebeat, en el servidor origen el agente ligero monitorea activamente los archivos de registro de seguridad generados por auditd y los accesos del servidor web Apache. Luego Filebeat reenvía las líneas de texto hacia Logstash, el cual actúa como motor de procesamiento, mediante el uso de filtros Grok, Logstash se encarga de parsear y estructurar tanto el formato complejo nativo de auditd como el formato de Apache. Una vez que los datos han sido estructurados, Logstash los envía a Elasticsearch, el cual los indexa en tiempo casi real, haciéndolos disponibles para consultas complejas y almacenándolos de manera segura mediante autenticación básica. Finalmente, Kibana se conecta con Elasticsearch para consumir los índices, permitiendo la creación de dashboards interactivos de seguridad y proporcionando la interfaz para la ejecución del módulo de Machine Learning (Elastic ML) para la detección de anomalías.

El despliegue de la arquitectura ELK mediante los contenedores presenta desafíos específicos respecto a la gestión de memoria en el laboratorio. En la configuración, la máquina virtual cuenta con un total de 15GB de memoria RAM disponible.



	total	used	free	shared	buff/cache	available
Mem:	15Gi	9.9Gi	4.7Gi	1.7Mi	1.3Gi	5.7Gi
Swap:	0B	0B	0B			

Debido a que el motor de Elasticsearch está basado en la Máquina Virtual de Java (JVM), su rendimiento depende críticamente de la correcta asignación de memoria. Para evitar escenarios de agotamiento de memoria al competir por recursos con el sistema operativo y los otros contenedores del pipeline, fue estrictamente necesario ajustar el heap size de la JVM. Se aplicó la recomendación técnica estándar de la industria de asignar exactamente la mitad de la RAM disponible al heap de Elasticsearch, configurando las variables de entorno "ES_JAVA_OPTS=-Xms7g -Xmx7g".

6.2 Implementación por componente

Los servicios principales se configuraron y desplegaron mediante Docker Compose, desplegando las versiones 8.12.0 de Elasticsearch, Logstash y Kibana en una misma red virtual. En el archivo de configuración docker-compose.yml se establecieron las dependencias de arranque y se expusieron los puertos estándar 9200, 5044 y 5601. Adicionalmente, se configuró el límite de memoria dinámica de la JVM para Elasticsearch a 7GB, asegurando la estabilidad del motor de indexación frente a la alta carga de logs que se podrían generar.

Para cumplir con los requisitos de seguridad de la arquitectura, se habilitó la autenticación nativa del stack. Inicialmente se definió una contraseña de arranque mediante la variable de entorno ELASTIC_PASSWORD. Posteriormente, para generar de forma segura las credenciales de los usuarios internos del sistema, como

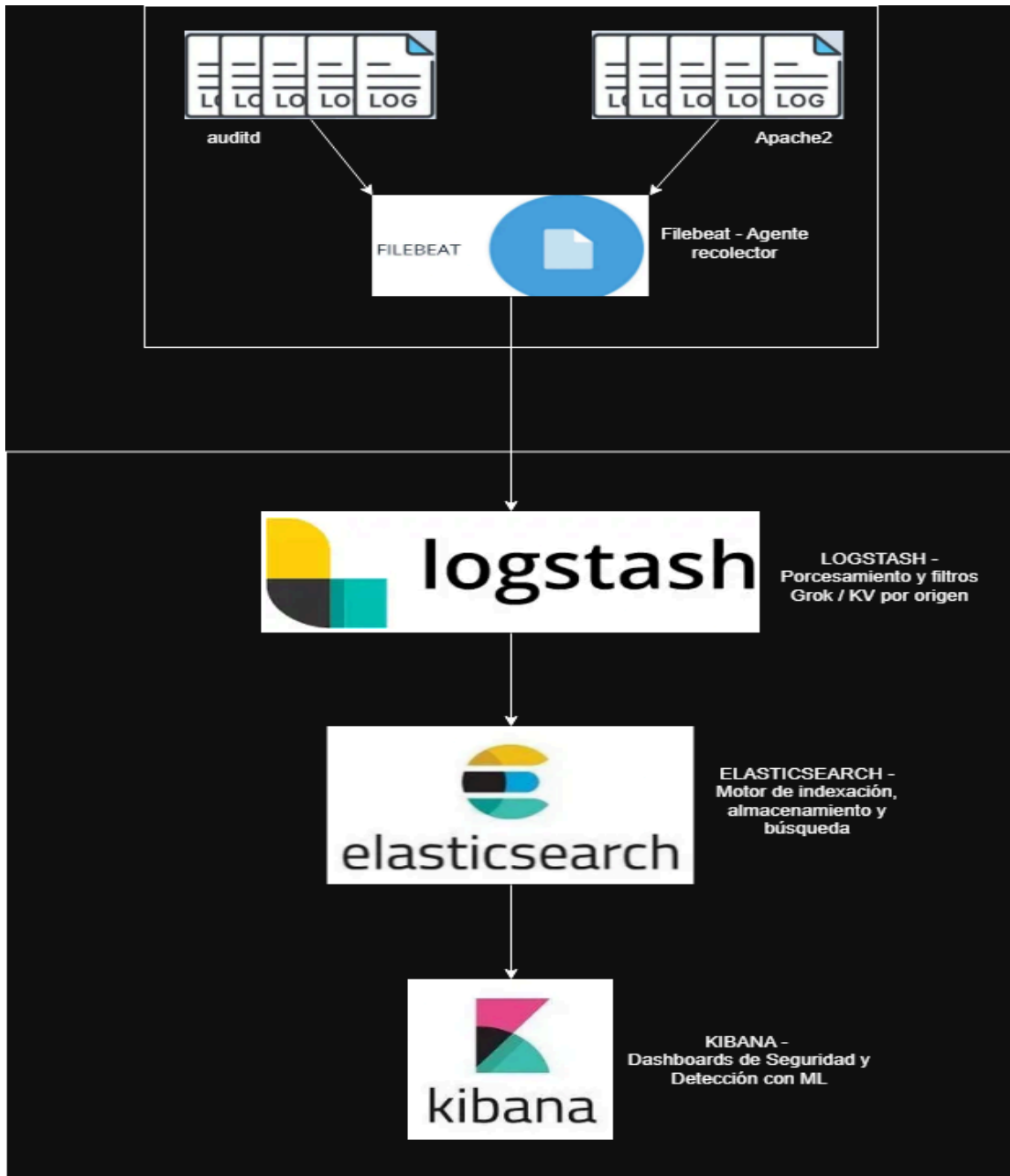
kibana_system, se accedió a la consola interactiva del contenedor de Elasticsearch y se ejecutó la utilidad de configuración de contraseñas:

```
docker exec -it elasticsearch sh
```

```
elasticsearch-setup-passwords interactive
```

La recolección de los logs en el servidor origen se delegó al agente Filebeat, cual fue configurado para leer los archivos locales de auditd y Apache, etiquetando cada flujo con un log_type específico y enviándolos por el puerto 5044. En el nodo central, Logstash recibe estos datos y aplica un procesamiento condicional basado en el tipo de log. Para resolver la complejidad estructural de auditd, se implementó un patrón grok que extrae el tipo de auditoría y la marca de tiempo, seguido de un filtro kv (Key-Value) que parsea automáticamente el resto de los campos dinámicos. Para los registros de Apache, se utilizaron los patrones estándar COMBINEDAPACHELOG para los accesos y una expresión regular a medida para los errores. Finalmente, los datos transformados son enviados a índices dinámicos en Elasticsearch según su fecha de origen.

Para complementar la arquitectura de recolección en el servidor origen, el agente Filebeat fue configurado mediante su archivo principal filebeat.yml. Se definieron dos entradas de tipo filestream encargadas de monitorear de forma continua los registros de auditoría del sistema operativo (/var/log/audit/audit.log) y los accesos y errores del servidor web (/var/log/apache2/access.log y error.log). Una decisión de diseño clave en esta configuración fue la inyección del campo personalizado log_type directamente en la raíz de cada evento. Esto es lo que hace permite que el nodo central de Logstash identifique el origen del dato de manera determinista y aplique el filtro Grok correspondiente. Finalmente, la salida estándar hacia Elasticsearch fue deshabilitada, delegando el reenvío de los paquetes exclusivamente al puerto 5044 de Logstash.



6.3 Análisis según modelo OSI o capas del SO

La solución implementada interactúa a través de múltiples capas, combinando el análisis a nivel de arquitectura del sistema operativo y el modelo de redes (OSI). A continuación, se detallan las capas involucradas:

Capa del sistema operativo (Espacio de Kernel y Espacio de Usuario) el **rol** que tiene en el proyecto es de generación y captura de eventos de seguridad a bajo nivel en el servidor origen. Los **mecanismos** utilizados son las llamadas al sistema, auditd en espacio de usuario y lectura de archivos mediante Filebeat. Una **amenaza** crítica

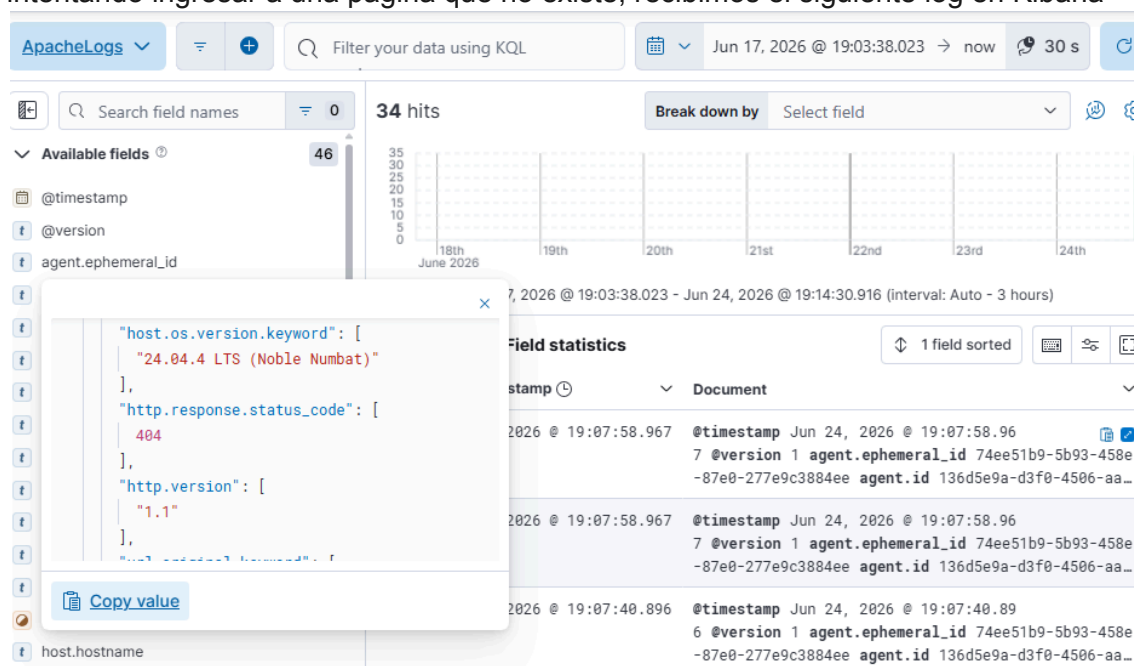
en esta capa es la manipulación de registros, donde un atacante con privilegios elevados podría borrar la evidencia local de los búferes antes de que sea almacenada de forma segura. La **mitigación** implementada consiste en el uso del agente Filebeat para extraer y reenviar los registros inmediatamente hacia un servidor centralizado externo, reduciendo la ventana de tiempo y evitando la pérdida de datos ante ataques locales o alta carga del sistema.

También nos encontramos con la **capa de aplicación (7)** respecto a la exposición de servicios web críticos y provisión de interfaces para la visualización de datos de seguridad. Los **protocolos** utilizados son HTTP y HTTPS (operando sobre el servidor Apache y la interfaz Kibana). Como **amenaza** tenemos la exposición a ataques de reconocimiento, escaneos de vulnerabilidades, denegación de servicio o fuerza bruta sobre rutas web, intentando generar errores HTTP 404 o 500 controlados. La **mitigación** aplicada en la arquitectura es de observabilidad y detección, se ingieren los logs y se procesan a través de Machine Learning para descubrir picos anómalos o secuencias de peticiones maliciosas que un administrador pasaría por alto.

Por otro lado, tenemos involucradas a las **capas de Transporte y Red (3 y 4)**, en la transferencia continua, estructurada y confiable de los flujos de registros desde los agentes locales hacia el pipeline centralizado, y comunicación interna de los nodos. Utilizamos los **protocolos** TCP e IP, utilizando puertos específicos para la ingesta y consulta de datos (Puerto 5044 para Logstash, 9200 para Elasticsearch y 5601 para Kibana). Respecto a las **amenazas y mitigaciones** tenemos la interceptación de tráfico de logs o manipulación de los datos en tránsito. Como mitigación principal en el entorno de despliegue, los servicios operan de manera aislada dentro de una red interna orquestada por Docker Compose, evitando la exposición directa de las bases de datos a interfaces públicas.

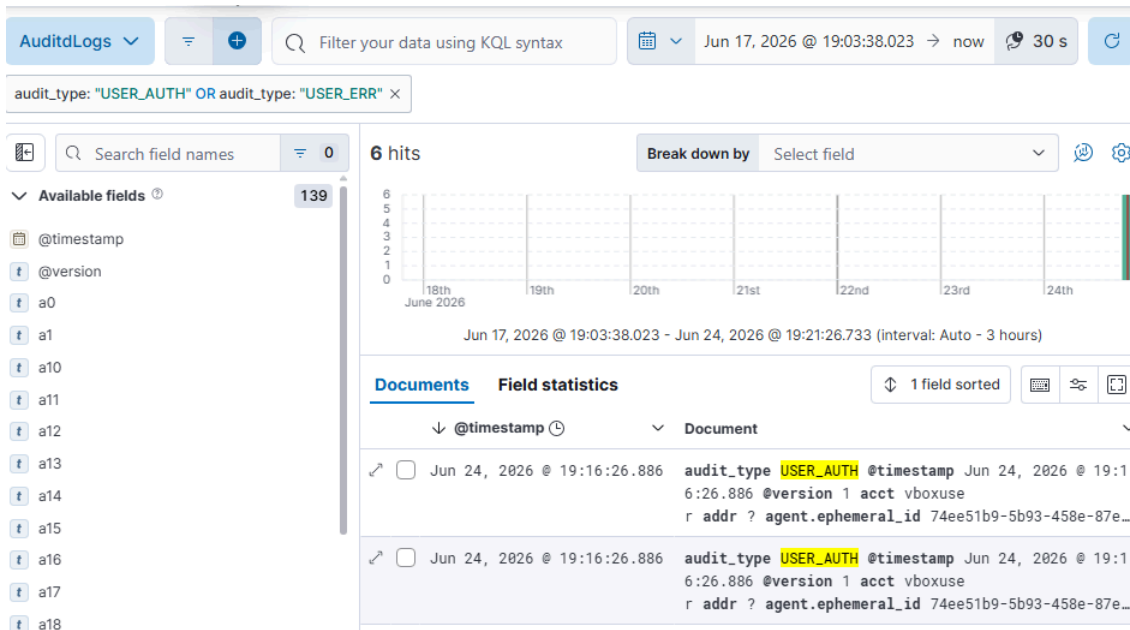
6.4 Pruebas y verificación

Al realizar una prueba del funcionamiento de la recolección de los de apache intentando ingresar a una página que no existe, recibimos el siguiente log en Kibana



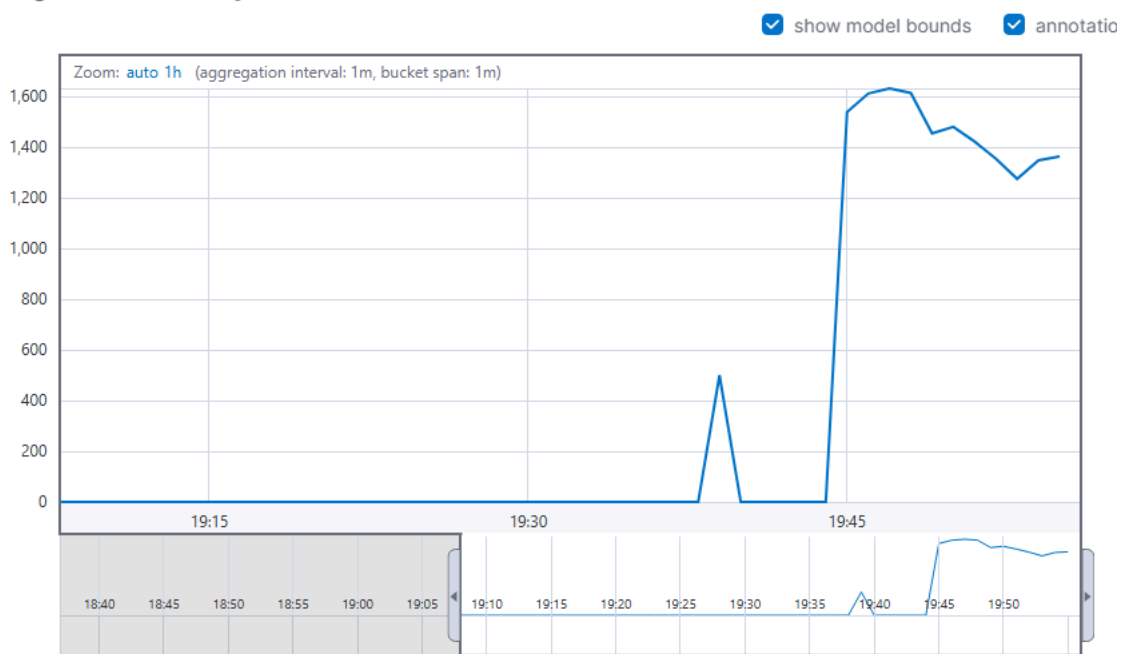
podemos observar que el código de estado es 404.

Por otro lado para poder comprobar la recolección de los logs cuando hay un intento de login se intenta acceder a un archivo no autorizado y el log llega exitosamente a Kibana



Respecto al módulo de Machine Learning podemos observar cómo reacciona ante un escaneo de puertos durante varios minutos

Single time series analysis of count



Time ↓	Level	Type	Entity ID	Message
Jun 24, 2026 @ 20:12:14.647	info	anomaly_detector	anomalias-apache	Datafeed has started retrieving data again
Jun 24, 2026 @ 20:09:14.463	warning	anomaly_detector	anomalias-apache	Datafeed has been retrieving no data for a while

7. Resultados y análisis

La evaluación de la arquitectura implementada nos dió resultados altamente positivos respecto a la eficiencia en la monitorización de seguridad. La contribución operacional principal de este trabajo reside en la drástica reducción del tiempo de respuesta ante incidentes.

Durante las pruebas, el análisis manual tradicional utilizando herramientas de consola como asearch y la inspección secuencial de archivos de texto requirió un esfuerzo de búsqueda estimado en más de 15 minutos continuos para lograr correlacionar los accesos denegados a /etc/shadow y la ejecución del escaneo de puertos, este tiempo fue la totalidad de búsqueda de varios eventos inyectados, sin embargo, al inyectar un evento a la vez el tiempo estimado de búsqueda manual es de aproximadamente 2 minutos. En contraste, el pipeline ELK centralizado permitió identificar, relacionar y visualizar la totalidad de los eventos maliciosos en aproximadamente 35 segundos, mediante la simple aplicación de filtros y consultas (KQL) sobre el índice dinámico, algunos de los filtros utilizados fueron “audit_type is ‘x’” o “res is ‘x’” dependiendo de la búsqueda que estamos realizando, al inyectar los eventos conscientemente la búsqueda es mas rapida ya que elasticsearch mediante el índice invertido tiene una alta velocidad, sin embargo, de forma manual conocemos el evento que estamos buscando pero debemos revisar la lista de logs en la terminal siendo no tan amigable con el analista debido a que la estructura es distinta a la que encontramos en Kibana. Los tiempos de búsqueda fueron tomados mediante un cronómetro para obtener una mayor precisión.

Adicionalmente, el módulo de Machine Learning fue capaz de identificar los eventos generados (como la ráfaga de actividad producida por Nmap o los intentos de sudo) como anomalías estadísticas, destacándose automáticamente sin necesidad de que un operador humano supiera exactamente qué buscar de antemano.

7.1 Comparación con los objetivos

Evento Inyectado	¿Detectado por ELK?	Tiempo de detección	¿Detectado manualmente?	Tiempo manual
Acceso a /etc/shadow	Si	40 seg.	Si	2:10 min.
Escaneo de puertos Nmap	Si	22 seg.	Si	2.05 min.
Ejecución de sudo no autorizado	Si	36 seg.	Si	2.36 min.

Acceso /etc/shadow:

- ELK Stack: Media de 30.20 seg., desvío estándar de 7.60 seg. en 5 repeticiones
- Detección manual: Media de 1.41 min (84.48 seg.), desvío estándar 0.41 min (24.79 seg.), en 5 repeticiones.

Escaneo de puertos Nmap:

- ELK Stack: Media de 28.40 seg., desvío estándar de 5.32 seg. en 5 repeticiones
- Detección manual: Media de 1.66 min (99.60 seg.), desvío estándar 0.40 min (24.13 seg.), en 5 repeticiones.

Ejecución de sudo no autorizado:

- ELK Stack: Media de 32.20 seg., desvío estándar de 4.44 seg. en 5 repeticiones
- Detección manual: Media de 1.46 min (87.84 seg.), desvío estándar 0.52 min (31.03 seg.), en 5 repeticiones.

Objetivo específico	Resultado obtenido	Estado
Desplegar una arquitectura centralizada basada en el Stack ELK mediante contenedores.	Se implementó exitosamente Elasticsearch, Logstash y Kibana mediante Docker Compose, garantizando el aislamiento y asignado correctamente los recursos de JVM	Cumplido
Normalizar e ingesta logs de SO y servicios críticos en tiempo real.	Filebeat y Logstash lograron parsear estructuralmente los registros complejos de auditd y Apache mediante el uso eficaz de filtros Grok.	Cumplido
Automatizar la detección de amenazas mediante visualización y Machine Learning	Se diseñaron dashboards interactivos que reflejos los ataques, y el modelo analítico detectó las anomalías volumétricas generadas en las pruebas.	Cumplido

7.2 Limitaciones identificadas

La arquitectura de Elasticsearch requiere una asignación sustancial de memoria (Heap de la JVM) para evitar caídas (OutOfMemoryError). En entornos de laboratorio o servidores de bajos recursos, esta dependencia limita la escalabilidad horizontal de la herramienta frente a un tráfico masivo de logs.

Debido a que el entorno de pruebas es un laboratorio controlado que carece de un histórico de meses de actividad “normal” de usuarios, el modelo de Machine Learning es altamente sensible a cualquier variación. Esto puede desencadenar alertas por anomalías frente a tráfico que, en un entorno de producción real, sería controlado legítimo.

7.3 Propuestas de mejora y trabajo futuro

Como trabajo futuro, se propone la integración de un mecanismo de notificación activa utilizando herramientas como Elastic Watcher o ElastAlert. Actualmente la solución

requiere que el analista ingrese a los dashboards de Kibana para observar las anomalías detectadas. La implementación de alertas automáticas, vía correo electrónico o mensajería instantánea, cuando el modelo de Machine Learning identifique un patrón malicioso o cuando reglas específicas de auditoría sean vulneradas, permitiría transformar este sistema de observabilidad pasiva en un Sistema de Prevención y Respuesta (IPS) proactivo. Asimismo, se sugiere entrenar el modelo analítico durante un periodo mínimo de 30 días en un entorno productivo para refinar la línea base de comportamiento y mitigar falsos positivos.

8. Conclusiones y trabajo futuro

La presente investigación ha demostrado de manera concluyente que la implementación de una arquitectura centralizada basada en el stack ELK (Elasticsearch, Logstash y Kibana), en conjunto con algoritmos de Machine Learning, optimiza drásticamente el proceso de auditoría y análisis forense de seguridad. Respondiendo a la pregunta de investigación planteada, se comprueba que un pipeline ELK es capaz de centralizar, indexar y analizar eventos de seguridad provenientes de sistemas operativos y servidores web en tiempo casi real, superando ampliamente las limitaciones de escalabilidad y la fricción temporal inherentes al análisis manual con

Evidenció una reducción significativa en el tiempo de respuesta ante incidentes. Mientras que la inspección secuencial en registros crudos requiere una inversión de tiempo prolongada e ineficiente, el entorno ELK procesó, transformó y estructuró la información visualmente en cuestión de segundos. Asimismo, la incorporación del módulo de Machine Learning permitió automatizar la identificación de patrones anómalos, tales como escaneos de red, accesos no autorizados a archivos críticos y escaladas de privilegios. Esto ratifica que las tecnologías impulsadas por Inteligencia Artificial (IA) no solo mejoran la eficacia de las auditorías de ciberseguridad, sino que descubren patrones ocultos que un operador humano podría pasar por alto.

Queda establecido que estas herramientas automatizadas no buscan reemplazar al analista de seguridad, sino delegar el procesamiento masivo de datos a la máquina. Esto permite liberar la carga operativa del profesional humano, otorgándole el tiempo necesario para enfocarse en actividades de alto valor, como la evaluación de riesgos, el monitoreo de cumplimiento y la toma de decisiones estratégicas.

A partir de los resultados obtenidos y considerando la constante evolución de las amenazas cibernéticas, se proponen las siguientes líneas de trabajo e investigación futura. Extender la ingesta de datos incorporando registros provenientes de sistemas de detección de intrusos (IDS) como Suricata, o logs de autenticación (SSH), para lograr una cobertura integral y enriquecer el contexto del análisis forense. Evaluar e incorporar algoritmos más complejos de clasificación múltiple (como XGBoost o Redes Neuronales Profundas) que permitan no solo detectar la existencia de una anomalía de manera binaria, sino clasificar automáticamente el tipo específico de ataque basándose en las características del log. Evolucionar la arquitectura actual, que funciona como un sistema de observabilidad pasiva, hacia una plataforma de Prevención y Respuesta (IPS/SOAR). Esto implicaría el desarrollo de scripts que, al recibir una alerta del modelo de Machine Learning, ejecuten reglas de mitigación automáticas en los firewalls del servidor origen.

9. Referencias bibliográficas

Anjum, N., & Chowdhury, R. (2024). Revolutionizing Cybersecurity Audit through Artificial Intelligence Automation: A Comprehensive Exploration. *International Journal of Advanced Research in Computer and Communication Engineering*.

Pozzi, M. A. (2021). Evolución del Malware y Defensa ante Ataques Dirigidos a Infraestructuras Críticas [Trabajo Final de Carrera, Universidad Abierta Interamericana].

Robbani, F. D., Haryatmi, E., Riyadi, T.A., Supono, R. A., Kurniawan, A. B., & Rosdiana. (2025). Implementation of an Intrusion Detection System Using Snort and Log Visualization Using ELK Stack. *International Journal of Engineering, Science and Information Technology*.

Sekar, R., Kimm, H., & Aich, R. (2024). eAudit: A Fast, Scalable and Deployable Audit Data Collection System. *2024 IEEE Symposium on Security and Privacy (SP)*.

Skopik, F., Landauer, M., & Wurzenberger, M. (2021). Online Log Data Analysis With Efficient Machine Learning: A Review. *IEEE Security & Privacy*.

Yang, C. T., Chan, Y. W., Liu, J. C., Kristiani, E., & Lai, C. H. (2021). Cyberattacks Detection and Analysis in a Network Log System Using XGBoost with ELK Stack. *Soft Computing*.

<https://www.lv12.com.ar/ciberataques/argentina-registro-5700-millones-intentos-ciberataques-2025-n198199>

<https://github.com/Myxz/tp-final-sor2-E3>